

REPORTE DETALLADO DE PRUEBAS UNITARIAS (QA)

ASEGURAMIENTO DE CALIDAD DETERMINISTA Y OFFLINE (PYTEST)

Estudiante: Marcelo Gilardoni

Usuario de GitHub: gilardoni72

Este manual tecnico describe individualmente el funcionamiento, entradas, salidas y criterios de validacion de las 6 pruebas unitarias deterministas implementadas en pytest (test_trading_api.py). Estas pruebas garantizan el correcto funcionamiento de tu API de Trading de forma 100% offline antes del despliegue productivo.

1. Filosofia de Pruebas Unitarias Deterministas (Sin LLM)

En la ingenieria de software moderna, las pruebas unitarias deben ser rapidas, deterministas y completamente independientes de servicios externos de red. Es por ello que nuestra suite esta diseñada bajo el patron de 'Tool Correctness'.

Comprueba de forma logica el correcto enrutamiento de FastAPI, la tipacion con Pydantic, el bloqueo perimetral de seguridad y el comportamiento de resiliencia ACID en disco (mock_db.json) sin realizar una sola llamada pagada o lenta a Gemini en internet. Esto permite correr las pruebas en menos de 2 segundos de forma local.

2. Analisis Detallado de las 6 Pruebas Unitarias

1. test_health_endpoint (Prueba de Salud del Sistema)

Qué hace: Envía una solicitud GET / al servidor FastAPI.

Qué valida: Comprueba que el servidor responda de inmediato con un código HTTP 200 OK y el JSON {'status': 'online'}.

Por qué importa: Confirma que la infraestructura web básica esté levantada y responda adecuadamente.

2. test_explain_trading_endpoint_unauthorized_missing_header (Seguridad Faltante)

Qué hace: Envía una simulación POST al endpoint de trading sin la cabecera X-API-Key.

Qué valida: Que el servidor bloquee la llamada y retorne de inmediato un código HTTP 401 Unauthorized.

Por qué importa: Garantiza que el cortafuegos de tu API esté activo por defecto ante accesos anónimos.

3. test_explain_trading_endpoint_unauthorized_wrong_key (Seguridad Clave Incorrecta)

Qué hace: Envía una simulación POST colocando una clave incorrecta en la cabecera X-API-Key.

Qué valida: Que el servidor identifique que la clave no coincide y retorne de inmediato un código HTTP 401 Unauthorized.

Por qué importa: Asegura que el validador autentique estrictamente los tokens (no solo que existan, sino que coincidan).

4. test_explain_trading_endpoint_authorized_success (Flujo Exitoso Completo)

Qué hace: Envía una simulación de inversión válida con la clave de acceso correcta.

Qué valida: Que el servidor retorne HTTP 200 OK, realice los cálculos matemáticos de ROI y comisiones de forma exacta en Python, y enmascare los datos personales (María Gomez -> M**** G****).

Por qué importa: Es el Happy Path del sistema; valida que tus algoritmos de negocio y enmascaramiento funcionen de forma integrada.

5. test_explain_trading_endpoint_validation_empty_message (Validación de Datos)

Qué hace: Envía una petición con un mensaje de consulta vacío o lleno de puros espacios.

Qué valida: Que el controlador FastAPI intercepte la entrada inválida y retorne un código HTTP 400 Bad Request.

Por qué importa: Previene que la API envíe prompts vacíos a internet, evitando costos inútiles de red en Google.

6. test_explain_alias_endpoint_success (Prueba de Retrocompatibilidad)

Qué hace: Envía una consulta válida utilizando la ruta de compatibilidad de los laboratorios anteriores (/weather/explain).

Qué valida: Que el alias funcione de manera idéntica al endpoint principal, retornando un HTTP 200 OK con el reporte

financiero.

Por qué importa: Garantiza la retrocompatibilidad estricta con las llamadas automáticas del profesor sin romper sus integraciones.

Fin del Reporte de Pruebas Unitarias - Marcelo Gilardoni | Solutions Architect & SRE.